

Chapter 3: A Framework for System Design Interviews

Introduction

System design interviews are a key part of the hiring process, simulating real-life problem-solving scenarios. These interviews evaluate not just technical skills but also collaboration, communication, and the ability to handle ambiguous requirements.

This chapter introduces a **4-step framework** for navigating system design interviews effectively.

Step 1: Understand the Problem and Establish Design Scope

Key Objectives

- Clarify requirements and assumptions.
- Avoid jumping into solutions prematurely.
- Showcase critical thinking by asking good questions.

Approach

- **Ask Clarifying Questions:**
 - What are the most important features?
 - What scale does the system need to handle?
 - Are we building for web, mobile, or both?
 - Are there existing technologies or constraints?
- **Document Assumptions:** Write assumptions on a whiteboard or paper for reference.

Example

Problem: Design a news feed system.

Questions:

- Is it a mobile app, web app, or both?
- How many friends can a user have?
- Should the feed include images and videos?
- Is the feed sorted by reverse chronological order?

Step 2: Propose High-Level Design and Get Buy-In

Key Objectives

- Develop a high-level architecture.
- Collaborate with the interviewer to refine the design.

Approach

- **Draft a Blueprint:**
 - Use box diagrams for key components (e.g., clients, APIs, databases, caches, CDNs).
 - Treat the interviewer as a teammate to refine the design.
- **Perform Back-of-the-Envelope Calculations:**
 - Ensure the design can handle the scale constraints.
- **Walk Through Use Cases:** Identify edge cases and validate design assumptions.

Example

For a news feed system, divide the design into:

1. **Feed Publishing Flow:** Writing posts into databases and populating friends' feeds.
 2. **Feed Retrieval Flow:** Aggregating and displaying friends' posts in reverse chronological order.
-

Step 3: Design Deep Dive

Key Objectives

- Dive into critical components.
- Showcase depth of understanding and adaptability.

Approach

- **Prioritize Key Components:** Focus on areas most relevant to the problem.
- **Discuss Bottlenecks:** Identify potential performance issues and propose solutions.
- **Balance Detail:** Avoid over-engineering or unnecessary deep dives.

Example Topics

- **URL Shortener:** Focus on hash function design.
 - **Chat System:** Explore latency reduction and online/offline status handling.
 - **News Feed System:** Examine feed publishing and retrieval processes.
-

Step 4: Wrap-Up

Key Objectives

- Highlight areas for improvement.
- Recap the design and discuss follow-ups.

Approach

- **Identify Bottlenecks:** Discuss potential limitations and scaling strategies.
- **Summarize Design:** Recap major design decisions and trade-offs.
- **Propose Enhancements:**
 - How to scale from 1 million to 10 million users.
 - Error handling for server failures or network issues.

Best Practices

Dos

- **Ask Questions:** Clarify ambiguities before diving into solutions.
- **Communicate:** Share your thought process with the interviewer.
- **Iterate with the Interviewer:** Treat them as a collaborator.
- **Show Flexibility:** Suggest alternative approaches and refine your design.
- **Focus on Critical Components:** Prioritize key parts of the system.

Don'ts

- **Avoid Premature Solutions:** Don't design before understanding the requirements.
 - **Don't Go Silent:** Communicate regularly during the process.
 - **Avoid Over-Engineering:** Focus on practical, scalable solutions.
-

Time Management

Suggested Time Allocation (for 45-Minute Interviews):

1. **Understand Problem and Scope:** 3–10 minutes
 2. **High-Level Design and Buy-In:** 10–15 minutes
 3. **Deep Dive:** 10–25 minutes
 4. **Wrap-Up:** 3–5 minutes
-